

Sujet de Projet PAF 2026: *Xenon 2 : Megablast*

PAF 2025 - Master STL S2

Version du 05/03/2025



Ce document présente le sujet du projet de l'UE PAF 2025 et contient les attendus, la description du projet, une liste des extensions possibles et des consignes pour le rapport.

Ce document complète le sujet d'Examen Réparti 1 du 10 mars.

1 Objectifs et Rendus

1.1 Objectifs

L'objectif principal de ce projet est le développement d'un **jeu** vidéo **sûr**, dans le langage **Haskell**. L'évaluation portera sur ces trois points:

1. programmation **sûre**: les types et les fonctions associées du projet doivent toutes être accompagnées de propositions d'**invariants**, de **pré-conditions**, et de **post-conditions**. Les opérations des entités doivent toutes être testées en utilisant, de façon raisonnablement complète, les **tests unitaires** (*HSpec*) et les **tests basés sur les propriétés** (*Quickcheck*).
2. programmation **fonctionnelle**: le langage Haskell est imposé, l'application doit être écrite dans un style fonctionnel, une attention doit être donnée à la séparation de la logique (évolution de l'état du jeu) et les effets de bords (boucle de jeu, entrée/sortie, affichage). L'utilisation de spécificités du langage Haskell est encouragée: types définis comme sommes d'états remarquables, instanciation de classes de types, utilisation des structures algébriques (**Monoid**, **Functor**, **Applicative**, **Monad**) et de leur bénéfices (*do-notation* par exemple).
3. programmation **graphique**: le jeu doit utiliser la bibliothèque **Gloss** afin de proposer une expérience visuelle, intuitive et fluide pour le joueur. Le jeu doit avoir un intérêt ludique non-négligeable.

Le projet sera évalué sur ces trois critères, ce qui veut dire, entre autres, que le projet peut s'éloigner des caractéristiques proposées dans ce document, tant qu'il satisfait les deux critères. Aucune définition (en particulier, les définitions de l'ER1) n'est imposée.

1.2 Rendu

Le projet est à faire en binôme (monôme possible avec autorisation de l'équipe enseignante, trinôme non-autorisé). Le rendu consiste en un projet GitLab (gitlab.com) contenant un rapport. Les enseignants doivent y être ajoutés comme *maintainer*.

La deadline est le ???.

2 Description du Jeu

(repris du partiel 2026:)

Sorti en 1989, *Xenon 2: Megablast* (par *The Bitmap Brothers*) propose un jeu de tir à défilement vertical (*vertical scrolling shooter* ou *shoot'em up*) dans lequel la joueuse dirige un vaisseau spatial qui doit éliminer ses ennemis en évitant les tirs et les collisions.

Le vaisseau de la joueuse peut se déplacer sur l'écran (vers le haut, le bas, la gauche la droite) pour éviter les collisions avec des ennemis, des obstacles ou les deux murs présents à gauche et à droite de l'écran.

Le jeu comporte un défilement (*scrolling*) : à intervalle réguliers (plusieurs fois par seconde) les obstacles, les murs et le décor au fond de l'écran se décalent vers le bas, donnant l'illusion (par parallaxe) que le vaisseau de la joueuse est constamment en mouvement vers le haut.

En plus de se déplacer, le vaisseau de la joueuse est capable de tirer des projectiles qui détruisent les ennemis quand ils entrent en collision avec eux. Les ennemis eux-mêmes peuvent tirer des projectiles sur le vaisseau de la joueuse. A chaque collision avec un ennemi ou un projectile, la barre de vie de la joueuse décroît. Quand elle est vide, le nombre d'essais de la joueuse (initialement à 2) décroît et la joueuse recommence le jeu à un point de passage récent. Le jeu s'arrête (*game over*) quand la joueuse n'a plus d'essai disponible.

Le jeu comporte en outre : cinq niveaux, avec des ennemis plus difficiles à vaincre (*boss*) à la fin de chacun d'eux, des bonus lâchés par certains ennemis à leur mort qui améliorent le vaisseau de la joueuse (tir plus rapide, tir dans d'autres direction) quand celui-ci les récupère (en passant dessus avec son vaisseau), la possibilité pour la joueuse d'inverser momentanément le défilement de l'écran, la possibilité de jouer à deux simultanément, un système de score.

On pourra chercher sur le Web des documents plus ou moins théoriques décrivant ce jeu. Par exemple :

- page *Wikipédia*: https://fr.wikipedia.org/wiki/Xenon_2:_Megablast
- un portage *DosBox* du jeu : <https://classicreload.com/play/xenon-2-megablast.html>
- vidéo *Youtube* d'une partie : <https://www.youtube.com/watch?v=XUfwSHAKb18>

On pourra aussi s'intéresser à d'autres jeux des genres *shoot'em up* et *bullet hell* et s'en inspirer : *1942*, *DonPachi*, *Touhou Project*, *R-Type*, *Ikaruga*, *Jamestown*, *Gladius*, ...

3 Contenu et Extensions

3.1 Consignes

Pour obtenir une **note satisfaisante** à l'évaluation, il faut que le projet respecte les contraintes suivantes:

- implémentation des **fonctionnalités** de l'ER1 en Haskell + Gloss (ou une modification du jeu de contenu équivalent),
- rédaction de **propriétés** pre/post/inv pour les types et les opérations du jeu,
- test systématique des propriétés pre/post/inv incluant **une utilisation pertinente de tests basés sur les propriétés** (*Quickcheck*).
- quelques extensions basiques, afin de proposer un contenu ludique minimal.

Pour obtenir la **note maximale**, le projet doit inclure de multiples extensions parmi celles proposées ou non (c'est encore mieux si elles sont originales), une utilisation pertinente **du test basés sur les propriétés** et des **classes algébriques**.

La motivation des extensions est importante: il faudra montrer ce qu'elles apportent au jeu, mais surtout ce qu'elles apportent au projet: complexité de la rédaction de propositions, élégance du code fonctionnel

utilisé, défi pour obtenir des tests complets, ... Une extension qui ne contiendrait pas de contenu "logiciel sûr" (propositions et tests) ou de contenu "programmation fonctionnelle" (utilisation de constructions propres à la programmation fonctionnelle ou à Haskell pour la mettre en oeuvre) n'ajouterait rien à l'évaluation.

3.2 Tests Basés sur la propriété

Le projet doit contenir une utilisation pertinente de *QuickCheck*, conforme à celles proposées en TME. Un soin particulier sera apporté à l'efficacité des générateurs (cf. le sujet de TME sur l'utilisation de **suchThat**).

La rédaction et l'utilisation de générateurs complexes pour des types non-triviaux est fortement encouragée. Dans tous les cas, les tests doivent être décrits dans le rapport et la méthode de rédaction des générateurs doit y être explicitée.

3.3 Classes de types

Les projets doivent rendre explicite la maîtrise des classes de types :

- manipulation correcte, élégante et efficace de **Maybe**, **Either**, **IO**, ...,
- instanciation de classes algébriques par des types définis dans le projet (par exemple **instance Monad Machin where ...**) et utilisation des bénéfices apportés par l'instanciation (primitives, *do-notations*, compositions, ...)

La programmation "avancée" avec les algébriques (rédaction de *transformers*, ...) est attendue pour une évaluation maximale.

3.4 Simplifications

Des simplifications du sujet proposé dans l'ER1 sont possibles (si elles sont compensées par des extensions de difficulté équivalente) :

- simplification des hitbox à des rectangles uniquement,
- suppression du caractère paresseux des murs gauche et droite,
- suppression de la violence (ennemis, tirs, ...),
- ...

3.5 Extensions

Voici quelques extensions possibles renforçant l'intérêt du projet en tant que "jeu vidéo" (on rappelle que les extensions originales sont encouragées). Le nombre d'extensions ajoutées au projet importe moins que leur taille, leur pertinence et leur intégration dans le programme du cours (propriétés, tests, algébriques, ...) :

- Un mode *multijoueur* (sur le même clavier, ou à distance ...) coopératif.
- Des *ajouts de gameplays* (inspirés ou non de jeux existants) : ennemis avec des comportements différents, bonus de vitesse, armes différentes, ...
- Le système d'inversion de défilement du jeu originel.
- Une version complètement procédurale du jeu (type *roguelite*).
- Une version en *3D* du jeu (soit discrète, soit continue).

4 Rapport

Le rapport de projet se compose de quatre parties:

- un **manuel** d'utilisation succinct expliquant comment tester le jeu depuis le projet *stack*.
- une liste exhaustive des **propriétés** (invariants/pré-conditions/post-conditions) implémentées dans le projet, classées par types / opérations et contenant une brève description de ce que vérifie la propriété.
- une description des **tests** implémentés par le **Spec.hs** du projet.

- un **rapport** proprement dit, qui décrit le projet, les extensions choisies et leur implémentation et qui, en plus, explicite les points importants de l'implémentation, en mettant en évidence du code, des propositions, des tests pertinents (ou des bugs mis découvertes grâce à la méthode de développement sûre).

De manière générale, il est fortement conseillé de consigner dans le rapport (sous la forme d'un paragraphe) toutes les difficultés rencontrées dans le développement du projet et tous les *points forts* (tests basés sur la propriété, utilisation d'algébriques, invariants de type complexe, ...) du projet afin de les mettre en valeur pour l'évaluation.

5 Barème Prévisionnel

Cette suggestion de barème n'est pas opposable, il s'agit d'un exemple possible pour la notation du projet, destiné à faire comprendre les priorités d'évaluation:

- 4 points pour **le rapport**: le rapport doit être clair, lisible, et contenir les informations nécessaires (manuel, propositions, tests). De plus, le rapport doit présenter suffisamment de points intéressants du développement du projet.
- 4 points pour l'implémentation correcte d'un **jeu minimum** en Gloss.
- 2 points pour **les propositions**: la rédaction des invariants/pre-conditions/post-conditions est systématique, pertinente, et commentée.
- 2 points pour **les tests**: les tests sont complets, correctement structurés, utilisent la puissance du *property based testing* et commentés.
- 1 point pour l'utilisation pertinente **du test basé sur la propriété**.
- 2 points pour l'utilisation pertinente **des classes algébriques**.
- 5 points pour **les extensions**: un nombre suffisant d'extensions ont été choisies, implémentées dans un style fonctionnel et (autant que possible) "haskellien", augmentées de propositions et de tests adéquats (on rappelle qu'une extension qui n'est pas accompagnée de inv/pre/post et de tests n'est pas évaluée). La pertinence, la difficulté et l'originalité des extensions choisies sont prises en compte.